Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.

3. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = -\sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

4. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

5. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to [0,1].
5. Convert labels into one-hot vectors.

6. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Task 5: Model Evaluation

Compute

- Accuracy

- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

6. Source Code

Source: Github
Github Repository Link:
`https://github.com/ssvibitha/DeepLearningLab_2026-27/blob/main/Lab2_MLP/DL_Lab2.ipynb`

7. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

8. Results

Best Hyperparameters

Hidden Layers	3
Hidden Neurons	128
Learning Rate	0.001
Batch Size	32
Optimizer	rmsprop
Activation Function	tanh
Epochs	30
Dropout	0.2
Cross-validation Accuracy	0.8906000000000001
Testing Accuracy	0.8825

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.865900	0.882500
Precision	0.877705	0.882713
Recall	0.865900	0.882500
F1-score	0.863692	0.882355
Training Time	113.793586 seconds	153.884540 seconds

9. Plots with Inference

1. **Sample Images**

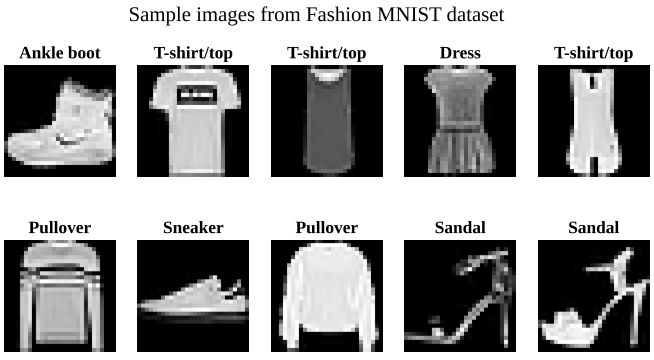

Figure 1: Sample Images

Sample Images Inference:
The grayscale sample images represent different fashion items along with the category they belong to. The wide range of data helps the model learn and distinguish between different clothing categories.

2. **Class Distribution**

Figure 2: Class Distribution

Class Distribution Inference:
The plot shows that there exists 10 different fashion item classes namely Tshirt/top, Trouser, Pullover, Dress, Coat, Sandal, Sneaker, Bag, Ankle Boot. This plot represents the class disttribution for training set. Each class contains 6000 samples.

3. **Baseline Model ConfusionMatrix**

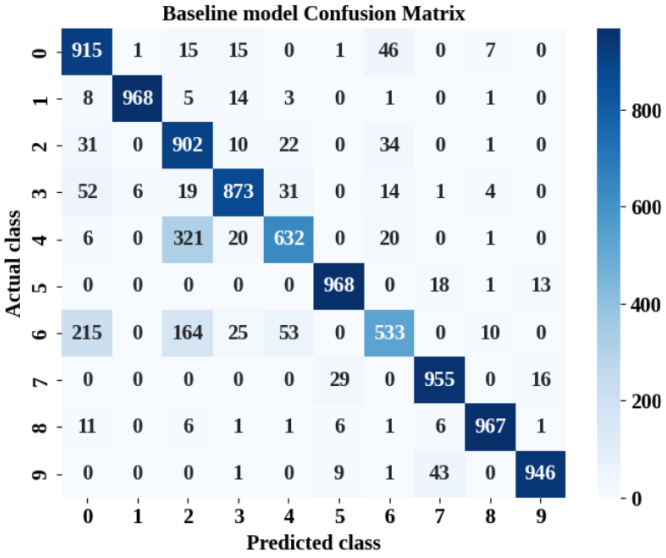

Figure 3: Baseline Model ConfusionMatrix

Baseline Model Confusion Matrix Inference:
The model correctly predicts most classes, performing best on classes 1, 5, 7, 8, and 9. It misclassifies class 4 as class 2 and class 6 as class 2 or class 0 often.

4. **Training and Validation Accuracy Vs Epoch**

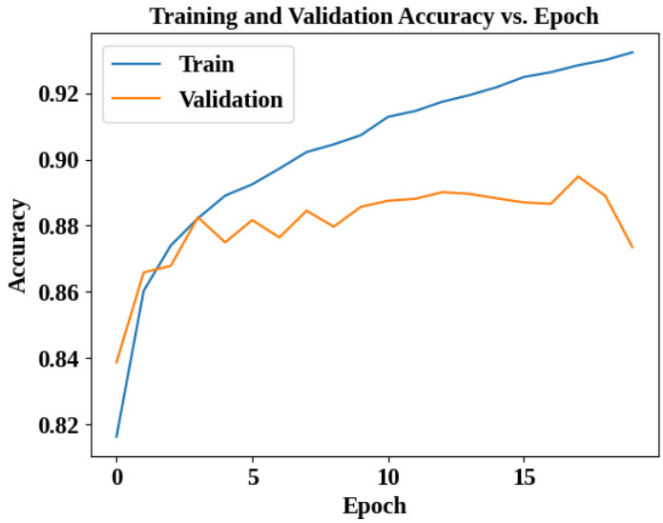

Figure 4: Training, Validation Accuracy Vs Epoch

Training, Validation Accuracy Vs Epoch Inference:
Training accuracy teadily increases throughout training, reaching over 93 percent showing the model is learning the training data well. Validation accuracy plateaus around 89 percent after epoch 5 and drops sharply at the end indicating the model stops improving on unseen data and begins to overfit.

5. **Training and Validation Loss Vs Epoch**

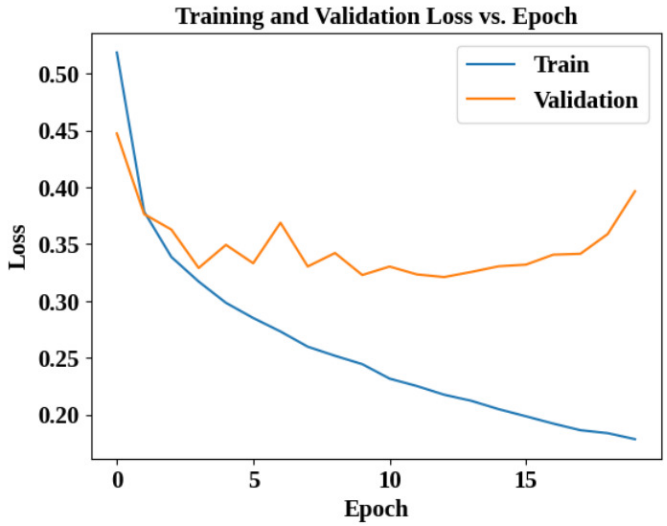

Figure 5: Training, Validation Loss Vs Epoch

Training, Validation Loss Vs Epoch Inference:
Training loss continuously decreases towards 0.18 showing consistent optimization on the training set. Validation loss decreases initially but hits a minimum around epoch 10 and 15 and begins rising sharply, clearly confirming overfitting in later epochs.

6. **Best Model Accuracy Comparison**

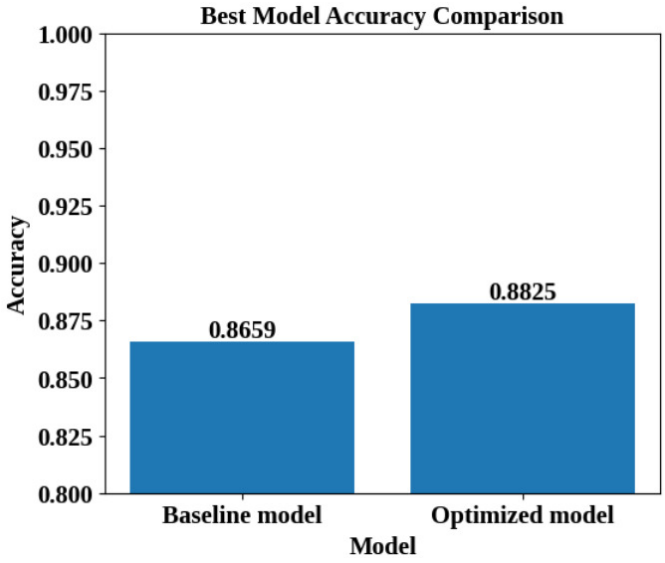

Figure 6: Best Model Accuracy Comparison

Best Model Accuracy Comparison Inference:
The optimized model achieved a higher accuracy of 88.25 percent compared to the baseline model's 86.59 percent. This indicates that hyperparameter tuning improved the model's classification performance.

10. Additional Task

Code the perceptron learning algorithm for the XOR gate using Python. Show the weights after each update. Plot the decision boundary after each weight update using Python's built-in packages. Analyze and identify the pattern that prevents the algorithm from converging.

1 Source Code

XOR using Single layer Perceptron
`https://github.com/ssvibitha/DeepLearningLab_2026-27/blob/main/Lab2_MLP/XOR_SLP.ipynb`

XOR using Multi layer Perceptron
`https://github.com/ssvibitha/DeepLearningLab_2026-27/blob/main/Lab2_MLP/XOR_MLP.ipynb`

2 Plots and Inference

1. **XOR using Single Layer Perceptron - Decision Boundaries**

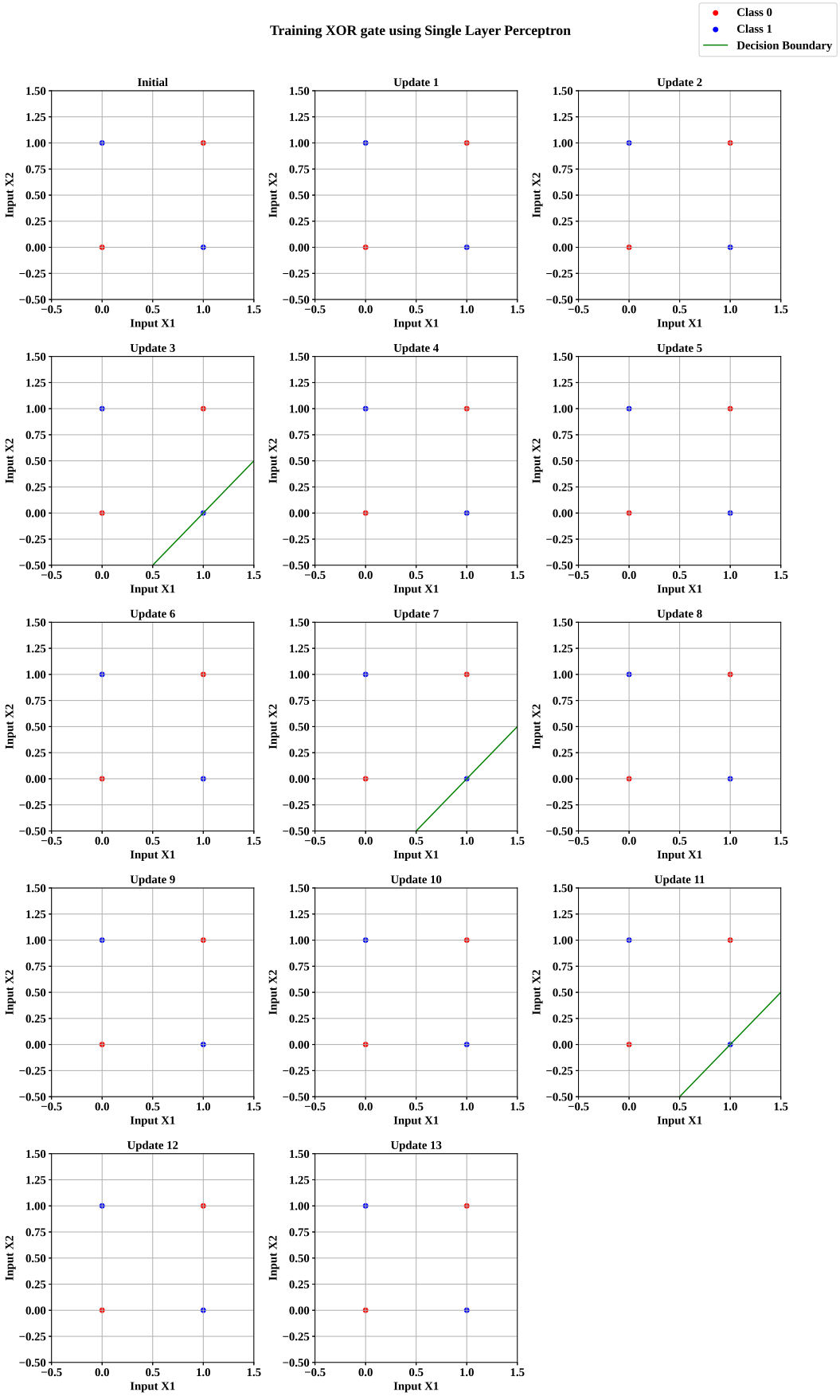

Figure 7: XOR using Single Layer Perceptron - Decision Boundaries

XOR using Single Layer Perceptron Inference:

- The figure shows the decision boundary at each update while training XOR using a perceptron.
- The XOR inputs belong to two classes:
  - **Class 0:** $(0, 0)$ and $(1, 1)$
  - **Class 1:** $(0, 1)$ and $(1, 0)$
- Across different updates, the decision boundary appears temporarily and keeps shifting or disappearing after further weight updates.
  - A decision boundary appears at some updates such as Updates 3, 7, 11, and 15.
  - At other updates, the decision boundary disappears because the weights are updated again.
  - This shows that the perceptron is continuously trying to correct misclassified points but cannot settle on a single boundary.
- The main reason for this behaviour is that XOR is not linearly separable.
  - The Class 0 points lie diagonally opposite each other.
  - The Class 1 points also lie diagonally opposite each other.
  - Therefore, no single straight line can correctly separate the two classes.
- Whenever the perceptron updates its weights to correctly classify one point, the new decision boundary may cause another point to become misclassified.
  - This results in another weight update.
  - The process continues repeatedly, causing the decision boundary to move back and forth instead of becoming stable.
- The plot shows that there is no stable final decision boundary even after several updates. The perceptron keeps changing its weights instead of converging to a solution.
- **Conclusion:**
  The pattern preventing the perceptron from converging is the repeated movement of the decision boundary caused by continuous misclassification. Since XOR requires a non-linear decision boundary while a perceptron can learn only a linear decision boundary. Hence the perceptron cannot successfully classify XOR.
  This demonstrates the limitation of a perceptron and shows that a multi-layer perceptron can help solve the XOR problem.

2. XOR using Multi Layer Perceptron - Decision Boundaries

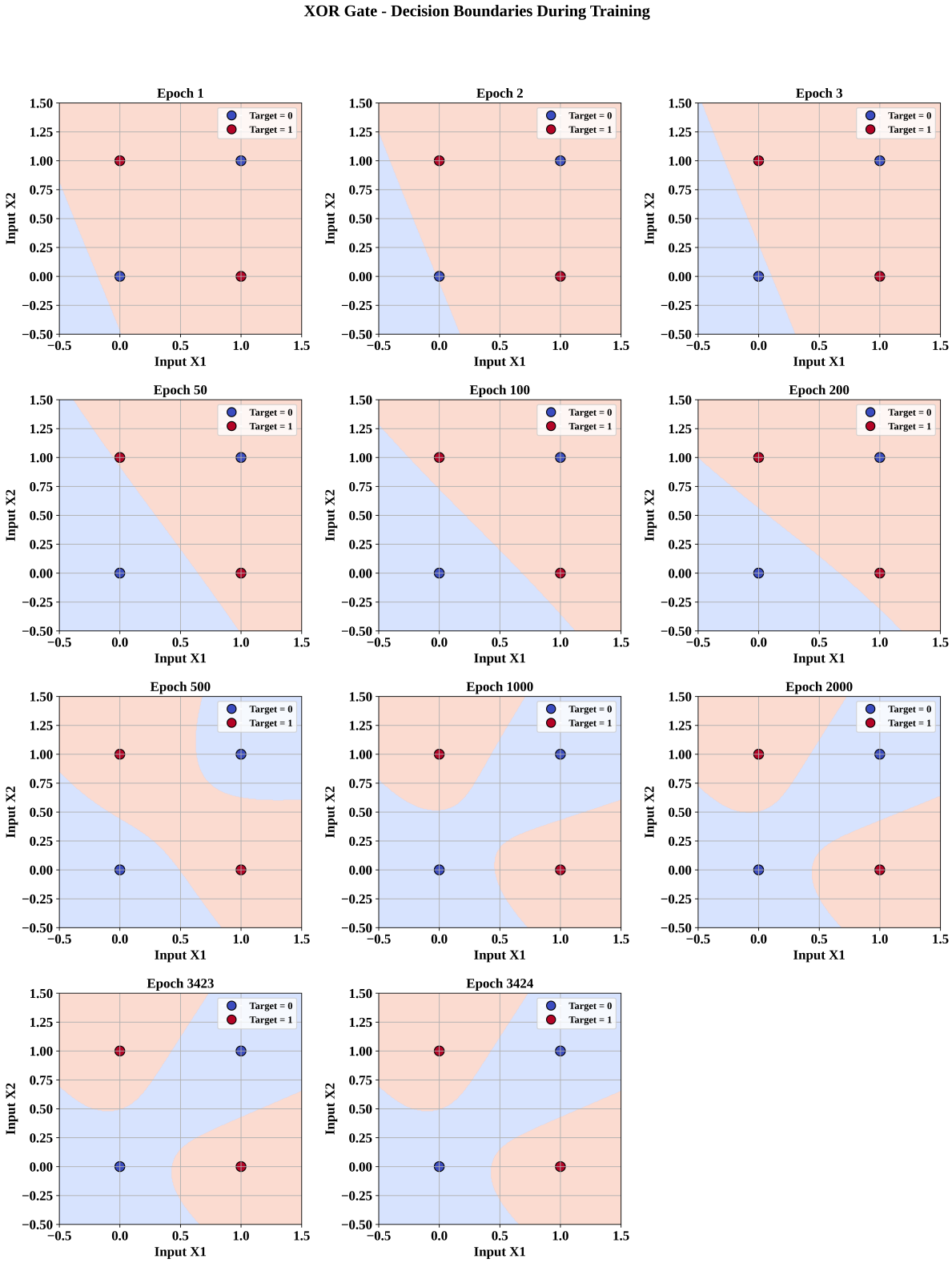

Figure 8: XOR using Multi Layer Perceptron - Decision Boundaries

XOR using Multi Layer Perceptron Inference:

- The plot shows the evolution of the decision boundary across multiple epochs for a XOR Gate trained using multi layer perceptron.
- During the early training, the decision boundary is a line that fails to separate the two classes.
- At epoch 500, we can see the two classes separated using a non-linear decision boundary. Subsequent epochs shows the shifting of decision boundary to maximise separation. Maximum separation between the two classes is achieved at epoch 3424.
- The XOR inputs are arranged diagonally. This makes the problem non-linear. Multi layer perceptron uses hidden layers and non-linear activation to achieve a non-linear decision boundary. The weights are adjusted using backpropagation in hidden layers.
- This shows that the multi layer perceptron can separate the two classes of XOR using a non-linear decision boundary.

11. Discussion

Answer the following:

1. **Which optimization method was used?**
   Randomized Search with Cross-Validation is used to find the best hyperparameters.

2. **Which hyperparameters were selected?**
   The selected hyperparameters are
   - Hidden Layers: 3
   - Hidden Neurons: 128
   - Learning Rate: 0.001
   - Batch Size: 32
   - Optimizer: RMSprop
   - Activation Function: tanh
   - Epochs: 30
   - Dropout: 0.2

3. **Did optimization improve performance?**
   Yes, after optimization the testing accuracy improved from 86.59 percent to 88.25 percent, along with improved precision, recall, and F1-score.

4. **Which hyperparameter had the greatest impact?**
   The optimizer RMSprop and learning rate of 0.001 had the greatest impact helping improve the model's performance.

5. **Compare Grid Search and Randomized Search.**
   Grid Search tests every possible combination of hyperparameters. Hence it is more computationally expensive. Randomized Search tests a random subset of combinations, and finds good results with less computation.

6. **Recommend the best MLP configuration.**
   MLP configuration with 3 hidden layers, 128 neurons, tanh activation, RMSprop optimizer, learning rate 0.001, batch size 32, dropout 0.2, and 30 epochs is recommended as it achieved the highest

testing accuracy of 88.25 percent.

12. Conclusion

The experiment successfully trained and evaluated an MLP classifier on the Fashion MNIST dataset. Hyperparameter tuning in training the Fashion MNIST dataset improved the model's accuracy and overall classification performance resulting in a more effective classifier. In addition to this, it was observed that the perceptron does not converge for the XOR gate because its classes cannot be separated using a single linear decision boundary.

13. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.